

Section 1.2

C++ Basic Language Features

1. Principles
2. Operators
3. Control structures
4. Variables and data types
5. Standard I/O streams
6. Functions
7. Function parameters
8. References
9. Programming conventions

1.2.1. Principles

- History of C++
 - C++ is a general-purpose object-oriented programming language
 - it was developed by Bjarne Stroustrup, starting in 1979
 - it was initially known as "C with classes", i.e. object-oriented C

Principles (cont.)

- Characteristics of C++
 - it provides all the standard high-level OO functionality
 - polymorphism
 - generic programming using *templates*
 - operator overloading
 - multiple inheritance
 - exception handling
 - it also allows calls to low-level C functions and system calls
 - memory management operations, basic I/O
 - bit-wise operations
 - C libraries for TCP/IP sockets, for POSIX threads, etc.

Terminology

- *Keyword*

- it's a word that has special meaning in a programming language
- examples: `while`, `int`, `return`

- *Identifier*

- it's a word that is *user-defined* (i.e. programmer-defined)
- examples: the names of variables, classes, functions

- *Expression*

- it's a sequence of keywords, identifiers, operators, constants
- an expression always returns a single value
- example: `a + b - 3 * c`

Terminology (cont.)

- *Statement*

- it's an expression that is terminated by a semi-colon
- examples:
 - `tmpValue = a + b - 3 * c; d * pow(g, a);`
- statements resolve to (i.e. "return") a single value
 - this value may be `void`, or it may be ignored by the program

- *Block*

- a sequence of statements between a pair of matching braces
- examples:
 - the body of a function, the content of a loop
 - each branch of an `if`-statement
 - free-floating blocks!

Terminology (cont.)

- *Scope* of an identifier:
 - the part of the program where the identifier can be used
- *Block scope*:
 - for example, a variable that is declared inside a block
 - this is called a *local variable*
 - any kind of block can have local variables
- *Global scope* (also called *file scope*)
 - for example, a variable that is declared outside of any block
 - this is called a *global variable*
 - industrial quality software **never** uses global variables
 - we will **never** use global variables in this course

Terminology (cont.)

- *Storage class* of a variable:
 - specifies both:
 - the lifetime of the variable
 - the area of memory where it's stored
 - *automatic* storage class
 - this is the default; variables are stored in the function call stack
 - these variables are created on declaration and destroyed on block exit
 - examples: local variables, function parameters
 - *static* storage class
 - variables are stored in the data segment (global memory)
 - they are created where *first* declared and destroyed at program termination
 - their values are “remembered” until the end of the program

1.2.2 Operators

- Types of operators in C/C++/Java:

- arithmetic: + - * / % ++ --

- relational: == != < > <= >=

- logical: && || !

- bitwise: ~ & | ^ >> <<

- assignment: = += -= *= /= %=

- conditional: ? :

- coding example <p1>

Operators (cont.)

- Characteristics of operators:
 - *arity*
 - this is the number of *operands* that an operator takes
 - operators can be *unary*, *binary*, or *ternary*
 - *precedence*
 - this is the order in which operators execute
 - remember BEDMAS?
 - *associativity*
 - this is the order in which operators of the *same precedence* execute
 - associativity can be left-to-right, or right-to-left

Operators (cont.)

- Very important operators in C++:
 - *stream insertion operator*
 - denoted as: <<
 - it adds a sequence of bytes to a stream
 - 1st operand is an output stream object, 2nd operand is the data to be output
 - *stream extraction operator*
 - denoted as: >>
 - it reads a sequence of bytes from a stream
 - 1st operand is an input stream object, 2nd operand is the variable to be read in
 - *scope resolution operator*
 - denoted as: ::
 - it's used to link an identifier to the scope that it belongs to
 - ... more on this later ...

1.2.3 Control Structures

- Conditional
 - example: `if-else`
- Selective
 - example: `switch`
- Iterative
 - example: `for, while, do-while`
- Jump
 - example: `break, continue`

Control Structures (cont.)

```
1 bool getArrayData(int* arr, int* num)
2 {
3     int newNum;
4     int currCount = 0;

5
6     cout << endl << "Enter the number of elements: ";
7     cin >> *num;
8     if (*num >= MAX_ARR_SIZE) {
9         cout << "Too many elements" << endl;
10        return false;
11    }

12
13    while (1) {
14        cout << "Enter the next number: ";
15        cin >> newNum;

16
17        if (newNum < 0) {
18            cout << "number cannot be negative" << endl;
19            continue;
20        }

21
22        arr[currCount] = newNum;
23        ++currCount;

24
25        if (currCount >= *num) {
26            break;
27        }
28    }
29    return true;
30 }
```

1.2.4 Variables and Data Types

- Characteristics of variables:
 - name
 - it's the identifier that the program uses to access or modify the content
 - value
 - this is the content of the variable
 - data type
 - it determines the number of bytes required in memory
 - location in memory
 - it's the address in memory where the value is stored

Variables and Data Types (cont.)

- Data types
 - primitive (built-in)
 - examples: `int`, `float`, `double`, `char`, `bool`
 - user-defined
 - defined by the programmer (that's us!)
 - these are our *classes*
 - memory address
 - example: pointers

Aggregate Data Types

- C-style `structs`

- in C++, they contain variables and functions, just like classes
- by default, their members are *public*
- this is **bad** software engineering



- Classes

- like Java, they contain variables and functions
- by default, their members are *private*
- using classes follows the *principle of least privilege*
- this is **good** software engineering

- **Do not** use `structs` in your assignments or tutorials

1.2.5 Standard I/O Streams

- C++ library stream objects
 - they are declared in the `iostream` library
 - they are encapsulated in the `std` namespace
- **cout**
 - object of class `ostream`, used for standard output
- **cin**
 - object of class `istream`, used for standard input
- **cerr**
 - object of class `ostream`, used for standard error
- **coding example <p2>**

1.2.6 Functions

- There are **two** kinds of functions in C++:
 - global functions
 - member functions
- *Global* functions
 - global functions are defined *outside* of any block
 - they can be called from any function and any class in the program
 - example of a global function: `main()`
 - example: all functions in C are global functions

Functions (cont.)

- Member functions

- member functions are defined inside a class
- they may be called on an object of that class
- *static* member functions can also be called on the class itself
 - this is a *different* kind of “static” from the type of storage class
- some OO languages call these functions “*methods*”
- the term *method* is **not** used in C++
- we use the term *member function* instead

Functions (cont.)

- Important terminology:
 - function *declaration*
 - this is the function prototype
 - function *definition* (or function *implementation*)
 - this is the function prototype **and** the code for that function
 - the code is also called the *body* of the function
- Member functions:
 - declaration and implementation are stored in **different files**

Function Design

- Characteristics of *correctly designed* functions:
 - they are *modular*
 - they “receive” data from the calling function, through parameters
 - they do some work and/or compute some result
 - they “give back” the results to the calling function
 - two ways to return the results: with a return value or using parameters
 - they are *single-purpose*
 - they have a single goal, they do **one** thing only
 - they use *abstraction* to hide their functionality
 - other functions know **what** function does, not **how** it does it
 - they must be *reusable*
 - in the same program, and in other programs

Function Design (cont.)

- Return values:
 - they are often used to indicate success or failure of a function
 - they can be used to return results from **very simple** functions
 - example: simple “getter” functions
 - normally, results are returned using an ***output parameter***
 - this allows multiple values to be returned to calling function
 - ... more on this coming up...

1.2.7 Function Parameters

- Parameters
 - they are treated as local variables inside a function
- Parameter modifiability:
 - it specifies if either the *value* or the *address* of a variable is used
 - usual situation:
 - calling function declares a variable and calls a function with it as parameter
 - the called function uses that variable to compute a result
 - can the called function make changes to the variable in calling function?
 - that depends on the *parameter modifiability*

Function Parameters (cont.)

- Two types of parameter modifiability:
 - pass-by-value
 - the parameter value is *copied* into the called function
 - the called function works on the local copy **only**
 - pass-by-reference
 - the *memory address* of the value is passed into the called function
 - this allow variables declared in the calling function to be modified
 - in C++, this can be done using ***pointers*** or ***references***

Function Parameters (cont.)

- Parameter direction
 - it specifies a parameter's role inside a called function
 - is the parameter *providing* information, or *returning* it, or both?
 - a parameter direction is specific to each parameter separately
 - parameter direction is *unrelated* to parameter modifiability
- Three types of parameter direction:
 - input parameter
 - output parameter
 - input-output parameter

Function Parameters (cont.)

- *Input parameter*

- this is a value that's needed by the function so it can do its work
- an input parameter is initialized by the calling function
- it is passed to the called function either by value or by reference

- *Output parameter*

- an output parameter is initialized by the called function
 - this is how results are correctly "returned" from the called function to the calling function
- it must be passed to the called function by reference

Function Parameters (cont.)

- *Input-output parameter*
 - this is both an input and an output parameter, at the same time
 - it is initialized by the calling function
 - the value is updated in the called function
 - it is passed back to the calling function
- **coding example <p3>**

1.2.8 References

- To pass parameters by reference:
 - the called function must be able to modify data stored in a calling function
 - one way to do this is with *pointers*
 - pointers are very powerful, but they can complicate the code
 - ... more on pointers later ...
 - an alternative to pointers: *references*
 - they are used for pass-by-reference
 - but they are *limited*
 - they are not a language feature in C, only in C++

References (cont.)

- What is a *reference*?
 - it's a **binding** between a new identifier and an existing variable
 - the new identifier becomes an *alias* for the variable it's bound to
 - declaring and initializing a reference creates a bond between:
 - the new identifier (the reference name), and
 - an existing variable
 - this bond is unbreakable for the scope of the new identifier
 - the most common usage is for passing parameters by reference

References (cont.)

- What is a reference *not*?
 - a reference is **not** a separate variable! it does not occupy memory
 - it must be bound to a variable *when it is declared*
 - the binding can never change to a different variable
- coding example <p4>

References (cont.)

- Most common usage of references: passing parameters
- `coding example <p5>`
- Pass-by-reference is always better than pass-by-value
 - passing parameters by value creates *copies* of data (this is bad)
 - we want to *avoid* copying objects
 - it is slow and bad software engineering in general
- Two ways to pass parameters by reference in C++
 - pass-by-reference using references
 - pass-by-reference using pointers
 - ... more on this in a later section ...

1.2.9 Programming Conventions

- Naming conventions
- Coding practices

Naming Conventions

- Naming conventions are *extremely* important
 - these are rules known to professional programmers
 - they help others understand your code quickly
 - they are an important *communication* tool between programmers
- Constants
 - names are all uppercase
 - example: **MAXNUM**
 - compound names are separated by underscores
 - example: **MAX_NUM**

Naming Conventions (cont.)

- Variables

- names begin with lowercase letter
 - example: `num`
- compound names use camel-case or underscore style
 - example: `numElements` or `num_elements`
- names must be descriptive without being overly long
 - use scope as context
 - example: inside `student` class, use `id` instead of `studentId`
- single letters can be used as temporary variables
 - example: loop counters `i`, `j`, `k`

Naming Conventions (cont.)

- Data types
 - names begin with uppercase letter
 - example: **Student** class
 - compound names use camel-case or underscore style
 - compound names for data types are less common
 - example: **BookArray** class

Naming Conventions (cont.)

- Functions

- names begin with lowercase letter
 - example: `sqrt()`
- compound names use camel-case or underscore style
 - example: `computeMaxValue()` or `compute_max_value()`
- names must be short and descriptive

Coding Practices

- There are many professional coding practices
- Some examples:
 - do design classes into object categories (entity, control, boundary)
 - do not use of **structs**, do use classes instead
 - do not use global variables
 - do not use global functions extensively
 - do not pass objects by value
 - this makes unnecessary copies of data, and slows down the program
 - do reuse code wherever possible
 - don't copy and paste code, call a common function instead
 - do perform basic error checking